Assignment 1: Smart Library Architecture Prototype

Saduakhas Olzhas - 26MD0341

1. Architecture Diagram and Explanation

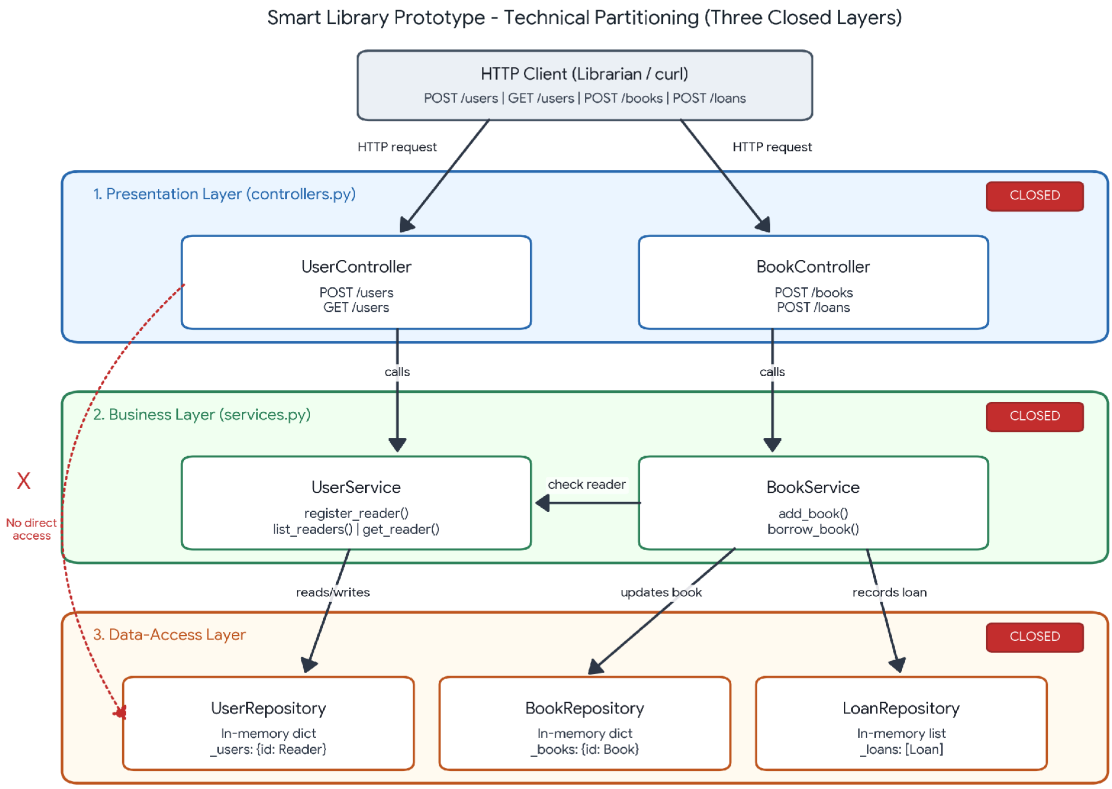


Figure 1
Three Closed Layers, Request Flow, and Permitted Dependencies

My prototype uses technical partitioning with 3 closed layers to create strict layer of isolation. In presentation layer, UserController and BookController parse HTTP JSON inputs and return status codes. In the business layer, UserService and BookService check inputs and apply lending rules, while UserRepository, BookRepository, and LoanRepository in data-access layer store records in memory. Marking all 3 layers as closed stops controllers from calling repositories directly. I have also put the loan logic inside BookService with reference to UserService so BookController only talks to only one service. On a POST /loans request, BookController passes reader_id and book_id into BookService.borrow_book(). That method checks the reader through UserService.get_reader() and looks up for book in BookRepository. If this book is available, BookService sets available to False in BookRepository and saves loan in LoanRepository.

2. Implementation Source Code

I was using Python 3 and built-in http.server module for my code below, so you no need extra pip packages for run mini projectt.

Source File models.py (Domain Entities)

```
from dataclasses import dataclass, asdict


@dataclass
class Reader:
    id: int
    name: str
    email: str

    def to_dict(self):
        return asdict(self)


@dataclass
class Book:
    id: int
    title: str
    author: str
    available: bool = True

    def to_dict(self):
        return asdict(self)


@dataclass
class Loan:
    id: int
    reader_id: int
    book_id: int

    def to_dict(self):
        return asdict(self)
```

Source File repositories.py (Data-Access Layer)

```
from models import Reader, Book, Loan


class UserRepository:
    def __init__(self):
        self._users = {}
        self._next_id = 1

    def create(self, name: str, email: str) -> Reader:
        reader = Reader(id=self._next_id, name=name, email=email)
        self._users[reader.id] = reader
        self._next_id += 1
        return reader

    def list_all(self) -> list:
        return list(self._users.values())

    def get_by_id(self, reader_id: int):
        return self._users.get(reader_id)


class BookRepository:
    def __init__(self):
        self._books = {}
```

```
        self._next_id = 1

    def create(self, title: str, author: str) -> Book:
        book = Book(id=self._next_id, title=title, author=author, available=True)
        self._books[book.id] = book
        self._next_id += 1
        return book

    def get_by_id(self, book_id: int):
        return self._books.get(book_id)

    def update_availability(self, book_id: int, available: bool):
        book = self._books.get(book_id)
        if book is not None:
            book.available = available
        return book


class LoanRepository:
    def __init__(self):
        self._loans = []
        self._next_id = 1

    def create(self, reader_id: int, book_id: int) -> Loan:
        loan = Loan(id=self._next_id, reader_id=reader_id, book_id=book_id)
        self._loans.append(loan)
        self._next_id += 1
        return loan
```

Source File services.py (Business Layer)

```
from repositories import UserRepository, BookRepository, LoanRepository


class LibraryError(Exception):
    def __init__(self, message: str, status_code: int):
        super().__init__(message)
        self.message = message
        self.status_code = status_code


class UserService:
    def __init__(self, user_repo: UserRepository):
        self.user_repo = user_repo

    def register_reader(self, name: str, email: str) -> dict:
        if not name or not email:
            raise LibraryError("Both name and email are required", 400)
        reader = self.user_repo.create(name=name.strip(), email=email.strip())
        return reader.to_dict()

    def list_readers(self) -> list:
        return [r.to_dict() for r in self.user_repo.list_all()]

    def get_reader(self, reader_id: int):
        return self.user_repo.get_by_id(reader_id)


class BookService:
    def __init__(self, book_repo: BookRepository, loan_repo: LoanRepository, user_service:
UserService):
        self.book_repo = book_repo
        self.loan_repo = loan_repo
        self.user_service = user_service

    def add_book(self, title: str, author: str) -> dict:
        if not title or not author:
```

```
            raise LibraryError("Both title and author are required", 400)
        book = self.book_repo.create(title=title.strip(), author=author.strip())
        return book.to_dict()

    def borrow_book(self, reader_id: int, book_id: int) -> dict:
        if not isinstance(reader_id, int) or not isinstance(book_id, int):
            raise LibraryError("reader_id and book_id must be integers", 400)

        reader = self.user_service.get_reader(reader_id)
        if reader is None:
            raise LibraryError(f"Reader {reader_id} does not exist", 404)

        book = self.book_repo.get_by_id(book_id)
        if book is None:
            raise LibraryError(f"Book {book_id} does not exist", 404)

        if not book.available:
            raise LibraryError(f"Book {book_id} is currently unavailable", 409)

        self.book_repo.update_availability(book_id, False)
        loan = self.loan_repo.create(reader_id=reader_id, book_id=book_id)

        return {
            "loan_id": loan.id,
            "reader_id": reader.id,
            "reader_name": reader.name,
            "book": book.to_dict()
        }
```

Source File controllers.py (Presentation Layer)

```
from services import UserService, BookService, LibraryError


class UserController:
    def __init__(self, user_service: UserService):
        self.user_service = user_service

    def create_user(self, payload: dict) -> tuple:
        try:
            reader = self.user_service.register_reader(
                name=payload.get("name", ""),
                email=payload.get("email", "")
            )
            return 201, reader
        except LibraryError as exc:
            return exc.status_code, {"error": exc.message}

    def get_users(self) -> tuple:
        readers = self.user_service.list_readers()
        return 200, {"readers": readers}


class BookController:
    def __init__(self, book_service: BookService):
        self.book_service = book_service

    def create_book(self, payload: dict) -> tuple:
        try:
            book = self.book_service.add_book(
                title=payload.get("title", ""),
                author=payload.get("author", "")
            )
            return 201, book
        except LibraryError as exc:
            return exc.status_code, {"error": exc.message}
```

```
    def create_loan(self, payload: dict) -> tuple:
        try:
            result = self.book_service.borrow_book(
                reader_id=payload.get("reader_id"),
                book_id=payload.get("book_id")
            )
            return 201, result
        except LibraryError as exc:
            return exc.status_code, {"error": exc.message}
```

Source File app.py (HTTP Router and Entry Point)

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer
from repositories import UserRepository, BookRepository, LoanRepository
from services import UserService, BookService
from controllers import UserController, BookController

user_repo = UserRepository()
book_repo = BookRepository()
loan_repo = LoanRepository()

user_service = UserService(user_repo)
book_service = BookService(book_repo, loan_repo, user_service)

user_controller = UserController(user_service)
book_controller = BookController(book_service)


class LibraryRequestHandler(BaseHTTPRequestHandler):
    def _send_json(self, status_code: int, body: dict):
         payload = json.dumps(body, indent=2).encode("utf-8")
         self.send_response(status_code)
         self.send_header("Content-Type", "application/json")
         self.send_header("Content-Length", str(len(payload)))
         self.end_headers()
         self.wfile.write(payload)

    def _read_json(self) -> dict:
        length = int(self.headers.get("Content-Length", 0))
        if length == 0:
            return {}
        raw = self.rfile.read(length).decode("utf-8")
        return json.loads(raw)

    def do_GET(self):
        if self.path == "/users":
            status, response = user_controller.get_users()
            self._send_json(status, response)
        else:
            self._send_json(404, {"error": "Route not found"})

    def do_POST(self):
        try:
            payload = self._read_json()
        except json.JSONDecodeError:
            self._send_json(400, {"error": "Invalid JSON body"})
            return

        if self.path == "/users":
            status, response = user_controller.create_user(payload)
            self._send_json(status, response)
        elif self.path == "/books":
            status, response = book_controller.create_book(payload)
            self._send_json(status, response)
        elif self.path == "/loans":
            status, response = book_controller.create_loan(payload)
```

```
            self._send_json(status, response)
        else:
            self._send_json(404, {"error": "Route not found"})

    def log_message(self, format, *args):
        return


if __name__ == "__main__":
    server = HTTPServer(("127.0.0.1", 8000), LibraryRequestHandler)
    print("Smart Library server running on http://127.0.0.1:8000")
    server.serve_forever()
```

3. Run and Test Instructions

Prerequisites for this mini project

Python >=3.9 installed on device. No third-party packages are needed, and you need place models.py, repositories.py, services.py, controllers.py, and app.py in same one folder.

Start the Application

```
python app.py
# Output: Smart Library server running on http://127.0.0.1:8000
```

How to Call Each Endpoint (curl Commands)

```
# 1. Register a reader (POST /users)
curl -i -X POST http://127.0.0.1:8000/users -H "Content-Type: application/json" -d "{\"name\":
\"Saduakhas Olzhas\", \"email\": \"ol_saduakhas@kbtu.kz\"}"

# 2. List registered readers (GET /users)
curl -i -X GET http://127.0.0.1:8000/users

# 3. Add a book (POST /books)
curl -i -X POST http://127.0.0.1:8000/books -H "Content-Type: application/json" -d "{\"title\":
\"Designing Data-Intensive Applications\", \"author\": \"Martin Kleppmann\"}"

# 4. Borrow a book (POST /loans)
curl -i -X POST http://127.0.0.1:8000/loans -H "Content-Type: application/json" -d "{\"reader_id\":
1, \"book_id\": 1}"
```

End-to-End Test Transcript (Request and Response Evidence)

Transcript below shows us complete test run against the running server: it is registering reader, listing readers, adding book, borrowing book (it updates its available field from true to false), and 2 rejected borrow requests (when the book is already borrowed and when the reader ID is unknown).

```
--- Step 1: Register a reader (POST /users) ---
Request:
POST /users HTTP/1.1
Content-Type: application/json

{"name": "Saduakhas Olzhas", "email": "ol_saduakhas@kbtu.kz"}

Response (HTTP 201 Created):
{
  "id": 1,
  "name": "Saduakhas Olzhas",
  "email": "ol_saduakhas@kbtu.kz"
}

--- Step 2: List registered readers (GET /users) ---
Request:
GET /users HTTP/1.1
```

```
Response (HTTP 200 OK):
{
  "readers": [
    {
      "id": 1,
      "name": "Saduakhas Olzhas",
      "email": "ol_saduakhas@kbtu.kz"
    }
  ]
}

--- Step 3: Add a book (POST /books) ---
Request:
POST /books HTTP/1.1
Content-Type: application/json

{"title": "Designing Data-Intensive Applications", "author": "Martin Kleppmann"}

Response (HTTP 201 Created):
{
  "id": 1,
  "title": "Designing Data-Intensive Applications",
  "author": "Martin Kleppmann",
  "available": true
}

--- Step 4: Borrow the book - valid request (POST /loans) ---
Request:
POST /loans HTTP/1.1
Content-Type: application/json

{"reader_id": 1, "book_id": 1}

Response (HTTP 201 Created - shows book available changed to false):
{
  "loan_id": 1,
  "reader_id": 1,
  "reader_name": "Saduakhas Olzhas",
  "book": {
    "id": 1,
    "title": "Designing Data-Intensive Applications",
    "author": "Martin Kleppmann",
    "available": false
  }
}

--- Step 5: Unsuccessful borrow case 1 - book is already unavailable (POST /loans) ---
Request:
POST /loans HTTP/1.1
Content-Type: application/json

{"reader_id": 1, "book_id": 1}

Response (HTTP 409 Conflict):
{
  "error": "Book 1 is currently unavailable"
}

--- Step 6: Unsuccessful borrow case 2 - unknown reader (POST /loans) ---
Request:
POST /loans HTTP/1.1
Content-Type: application/json

{"reader_id": 99, "book_id": 1}

Response (HTTP 404 Not Found):
{
  "error": "Reader 99 does not exist"
}
```

4. Architecture Quality Table and Reflection

Table 1

Quality Attribute	Rating (1-5)	Reason Tied to Prototype and Course Concepts
Modifiability	5 / 5	Closed layers create layer of isolation, so storage in repositories.py can be replaced without editing services.py or controllers.py
Scalability	2 / 5	All data stays in local Python dicts inside one process (architecture quantum of one), so running second server process would not share state
Reusability	4 / 5	UserService and BookService have 0 afferent coupling to HTTP classes and return plain dicts, so CLI or batch script can reuse them
Maintainability	5 / 5	Methods in services.py have high functional cohesion and low cyclomatic complexity, so tracing bug from controller down to repository is simple

Reflection

Grouping related methods into separate modules gives each class high functional cohesion and limits coupling across encapsulation boundaries to static connascence of name. Because whole prototype runs in one process with shared in-memory stores, it forms architecture quantum of the one. Main trade-off that all state resets on restart, and concurrent requests would risk dynamic connascence of timing through race conditions. If library later needed to handle thousands of a readers, I would replace all classes in repositories.py with PostgreSQL queries and lock book row inside of database transaction during borrow_book() so 2 desks cannot lend same copy.

5. Permitted AI-Use Note

AI use note: I was used Gemini Argon 4 during this assignment to review the lecture terms on connascence and closed layers and also to check my English grammar where needed.